LogPulse

Serverless QA & CI/CD Log Analyzer on Microsoft Azure

Project documentation

Danial Umer
Óbudai Egyetem
November 2026

Contents

1. Introduction 3
1.1 Goals 3
1.2 Scope 3
2. Architecture 3
2.1 Azure services used 4
2.2 Design decisions 4
3. Implementation 5
3.1 Log parsing 5
3.2 Azure Functions 6
3.3 Data model 6
3.4 Query API and dashboard 7
3.5 Security 7
4. Deployment on Azure 7
4.1 Problems during deployment 8
5. Testing and results 9
5.1 Unit tests 9
5.2 End-to-end test on Azure 9
5.3 Dashboard 13
6. Cost 15
7. Limitations and future work 15
8. Conclusion 16
Appendix A: Running the project 16
Appendix B: Repository structure 16

1. Introduction

Every CI/CD run produces a log, and when a build fails someone has to open that log and find out why. For a small project this is fine, but a real pipeline log is often several thousand lines long and mixes dependency installation, compiler output, test runner output, security scanner tables and runner messages. The actual reason for the failure is usually a handful of lines somewhere in the middle.

LogPulse automates this step. When a raw log file is uploaded to Azure Blob Storage, an event triggers a serverless Python function that parses the text, extracts the important error states (failed tests, build errors, crashes, security findings, leaked secrets and infrastructure problems) and stores a structured summary in Azure Cosmos DB. QA and DevOps engineers can then search these summaries through a small web dashboard and a REST API instead of reading the raw logs.

1.1 Goals

- Process logs automatically and event-driven: no polling, no manual step after the upload.
- Support the log formats of the common CI platforms and tools, not just one framework.
- Store results in a structure that can be filtered and searched (by pipeline, status, severity, category or free text).
- Run fully serverless, so the system costs nothing while idle.
- Avoid storing any keys or passwords in the application configuration.

1.2 Scope

The project covers the full path from upload to dashboard: the log parser, the Azure Function with its blob trigger, the Cosmos DB data model, a query API, a dashboard, the infrastructure as code and a deployment script. Six realistic sample logs from GitHub Actions, Azure Pipelines, GitLab CI and Jenkins were written to test the system end to end.

2. Architecture

Figure 1 shows the components and how a log moves through the system.

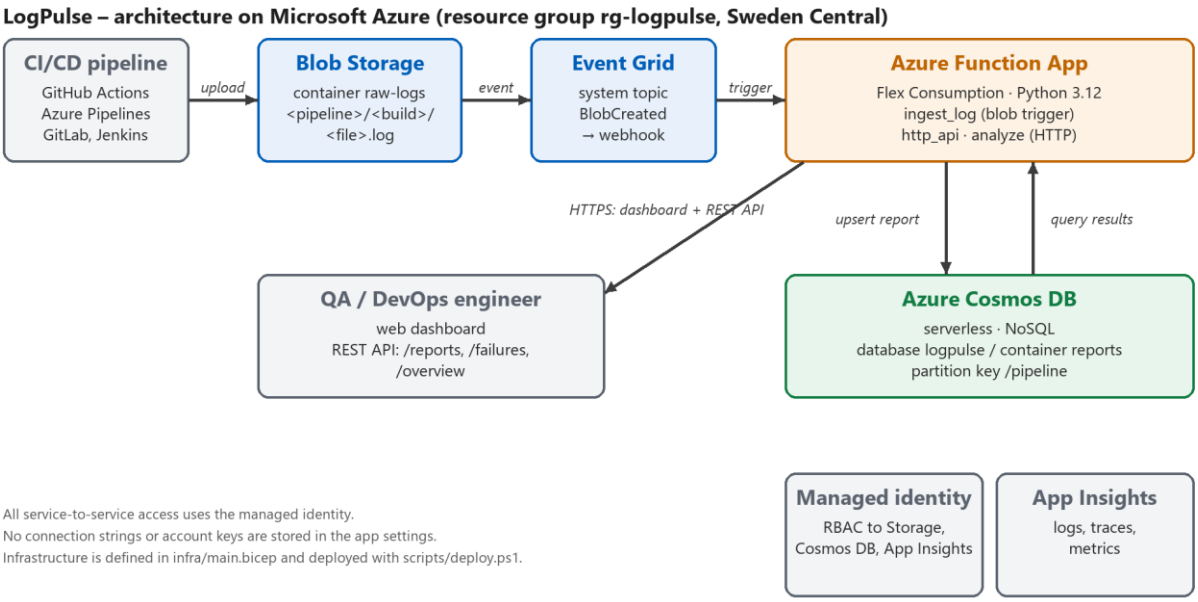


Figure 1: LogPulse architecture

The flow for one log file is:

1. A pipeline (or a user) uploads the log to the raw-logs container as <pipeline>/<build id>/<file>.log.
2. Blob Storage publishes a BlobCreated event to an Event Grid system topic.
3. The Event Grid subscription delivers the event to the ingest_log function within a few seconds.
4. The function downloads the blob, parses it and upserts one report document into Cosmos DB.
5. The dashboard and the REST API (the http_api function) query Cosmos DB and return the reports.

2.1 Azure services used

Table 1: Azure services used

Service	Resource	Purpose	Why this choice
Blob Storage	stlogpulse…	Receives raw logs in container raw-logs	Cheap, every CI system can upload to it
Event Grid	evgt-logpulse…	Turns BlobCreated into a webhook call	Push-based, triggers in seconds
Azure Functions	func-logpulse…	Parsing, HTTP API, dashboard	Flex Consumption plan, scales to zero
Cosmos DB (NoSQL)	cosmos-logpulse…	Stores one JSON report per log	Serverless capacity, flexible schema
Application Insights	appi-logpulse…	Function logs and metrics	Built-in integration with Functions
Managed identity	id-logpulse…	Authenticates the function to other services	No secrets in configuration

2.2 Design decisions

Event Grid instead of the polling blob trigger. The classic blob trigger scans the container periodically and can be several minutes late on a consumption plan. The Event Grid based trigger reacts to the

BlobCreated event directly, which fits the event-driven idea of the project and is also the only blob trigger type supported on the Flex Consumption plan.

Partition key /pipeline. The most common question is "show me the recent builds of this pipeline", so partitioning by pipeline keeps that query inside one partition. Searches across all pipelines are still possible as cross-partition queries.

Idempotent processing. Event delivery is at-least-once, so the same blob can be processed twice. The document id is derived from the blob path and the function uses upsert, so a duplicate event overwrites the report instead of creating a copy.

Parser independent of Azure. All logic lives in a normal Python package (logpulse). The Azure Function file only adapts the triggers to it. This made it possible to develop and test everything locally, including a local server that simulates the blob trigger with a watched folder.

3. Implementation

3.1 Log parsing

The parser first cleans every line: it removes ANSI colour codes, runner timestamps, GitLab section markers and progress bar redraws. Then it runs a set of detection rules over the lines. Each rule is a regular expression plus a small function that turns a match into a finding (category, severity, title, file, line number and so on). In total there are 44 rules and 8 secret patterns. Table 2 lists what is covered.

Table 2: Detection coverage

Category	Examples of supported tools / formats
Test failures	pytest, unittest, JUnit (Maven Surefire, Gradle), Jest, Go test, dotnet test, RSpec
Build errors	TypeScript tsc, javac / Maven, gcc / clang, Go, Rust, mypy, npm, pip, Docker BuildKit, make
Exceptions	Python tracebacks, Java uncaught exceptions, Node.js errors with stack trace, Go data races
Security	npm audit, Trivy, Snyk, Bandit, any CVE / GHSA / PYSEC / RUSTSEC id, insecure TLS settings
Secret leaks	AWS keys, GitHub and Slack tokens, private keys, Azure storage keys, JWTs, hard-coded passwords
Infrastructure	out of memory / exit 137, segmentation faults, disk full, timeouts, DNS and network errors
CI runner	##[error], ::error::, non-zero exit codes, GitLab "Job failed", Jenkins "Finished: FAILURE"

A few details turned out to matter more than expected:

- **Deduplication.** Tools often print the same error twice (for example BuildKit prints a failed step and then "failed to solve" with the same message). Every finding gets a fingerprint, and findings with the same fingerprint are merged into one with an occurrence count.
- **Fingerprints across builds.** For free-text messages, numbers and hashes are masked before hashing, so "timed out after 31s" and "timed out after 29s" are recognised as the same problem in different builds. Exact identifiers such as CVE ids and test names are not masked. My first version masked them too, which merged CVE-2024-45491 and CVE-2024-45492 into one finding; the unit tests caught this.
- **Secret redaction.** Secret detection runs before all other rules, and the secret value is replaced with ***REDACTED*** in the line. Because of that, a leaked key never reaches the database, not even in the context lines stored with a finding.

- **Test statistics.** The summary lines of the test runners (e.g. "2 failed, 45 passed, 1 error in 3.81s") are read separately, so each report also contains total, passed, failed and skipped counts.

The overall status of a log is failed if there is any high or critical finding or any failed test, unstable if the worst finding is medium, and passed otherwise.

3.2 Azure Functions

The Function App uses the Python v2 programming model. It contains three functions:

Table 3: Functions in the Function App

Function	Trigger	Description
ingest_log	Blob (Event Grid source), raw-logs/{name}	Parses a new log and upserts the report into Cosmos DB
http_api	HTTP GET, anonymous	/api/reports, /api/failures, /api/overview, /api/health and the dashboard
analyze	HTTP POST, function key	Parses a log sent in the request body, optionally stores it

```
@app.blob_trigger(arg_name="blob", path="raw-logs/{name}",
                  connection="AzureWebJobsStorage", source=func.BlobSource.EVENT_GRID)
def ingest_log(blob: func.InputStream) -> None:
    report = service.process_blob(blob.name, blob.read(), get_store())
    logging.info("Stored report %s/%s status=%s",
                 report["pipeline"], report["id"], report["status"])
```

If a log is larger than the configured limit (100 MB by default), the function logs the problem and returns without raising an error. Raising would only cause pointless retries for a file that can never be processed.

3.3 Data model

Each log becomes one JSON document in the container reports. A shortened example:

```
{
  "id": "1042-pytest.log",
  "pipeline": "api-service",           <- partition key
  "build_id": "1042",
  "status": "failed",
  "ci_platform": "github-actions",
  "tests": { "total": 48, "passed": 45, "failed": 2, "errors": 1, "skipped": 0 },
  "severity_counts": { "critical": 2, "high": 5, "medium": 1, "low": 1 },
  "top_finding": "Possible AWS access key exposed in log",
  "findings": [
    { "category": "test_failure", "severity": "high", "source": "pytest",
      "title": "tests/test_orders.py::test_create_order_total failed",
      "message": "assert 29.969999999999995 == 29.97",
      "line": 92, "context": [ "...lines around the failure..." ],
      "fingerprint": "5d0c6f2a91b4e0c7", "occurrences": 1 }
  ]
}
```

A Cosmos DB item can be at most 2 MB. For very noisy logs the report is shrunk before saving: first the context lines are shortened, then the least severe findings are dropped, and the document is marked findings_truncated. The context lines are also excluded from the indexing policy because nobody filters on them.

3.4 Query API and dashboard

The API supports filtering by pipeline, status, minimum severity, category, fingerprint, date and free text. Filters on findings are translated into a Cosmos DB SQL query with an EXISTS sub-query over the findings array. All values are passed as query parameters, never concatenated into the SQL text. The dashboard is a single HTML page without external libraries, served by the same function. It has three views: builds, individual findings, and recurring problems (findings with the same fingerprint in several builds).

3.5 Security

- The function uses a user-assigned managed identity for Blob Storage, Cosmos DB and Application Insights. The app settings contain only names and endpoints, no keys (Figure 8).
- The identity has only the roles it needs: Storage Blob Data Owner, Storage Queue and Table Data Contributor on the storage account, Cosmos DB Built-in Data Contributor on the Cosmos account and Monitoring Metrics Publisher on Application Insights.
- Public blob access is disabled and TLS 1.2 is the minimum version on all resources.
- The write endpoint (analyze) requires a function key; the read-only endpoints are anonymous for the demo.
- Secrets found in logs are redacted before storage (Section 3.1).

4. Deployment on Azure

All resources are described in one Bicep file (infra/main.bicep). The PowerShell script scripts/deploy.ps1 registers the needed resource providers, deploys the Bicep template, packages the code, deploys it to the Function App with a remote build, and finally creates the Event Grid subscription. The subscription has to be created after the code is running, because its webhook URL contains a system key of the blob extension that only exists once the function host has started.

```
az login
.\scripts\deploy.ps1 -Location swedencentral
.\scripts\upload-samples.ps1
```

Everything was deployed into the resource group rg-logpulse in the Sweden Central region (Figures 2 and 3).

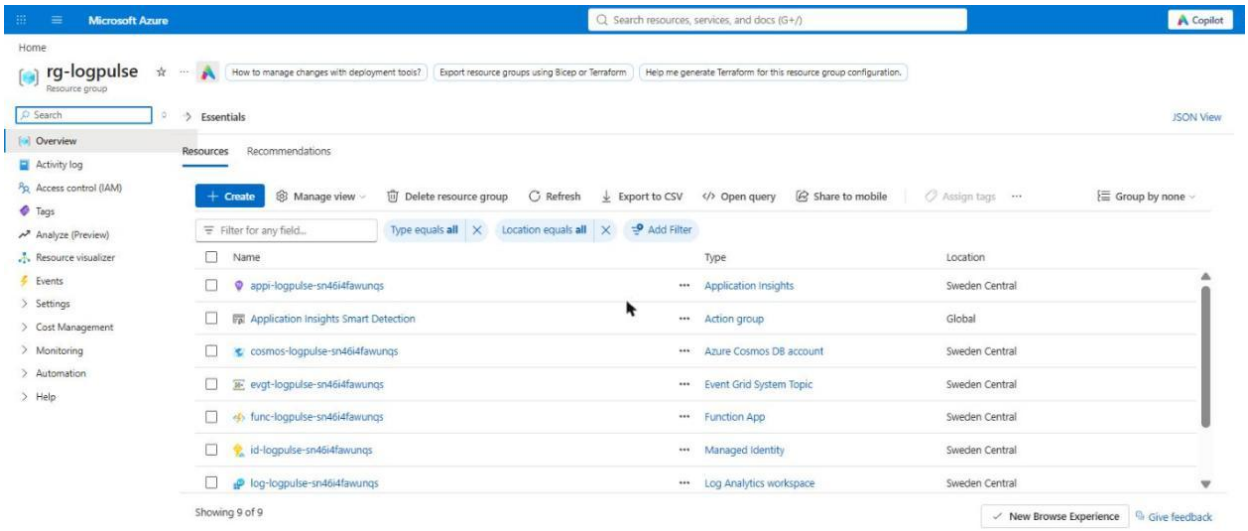


Figure 2: Resource group rg-logpulse with all deployed resources

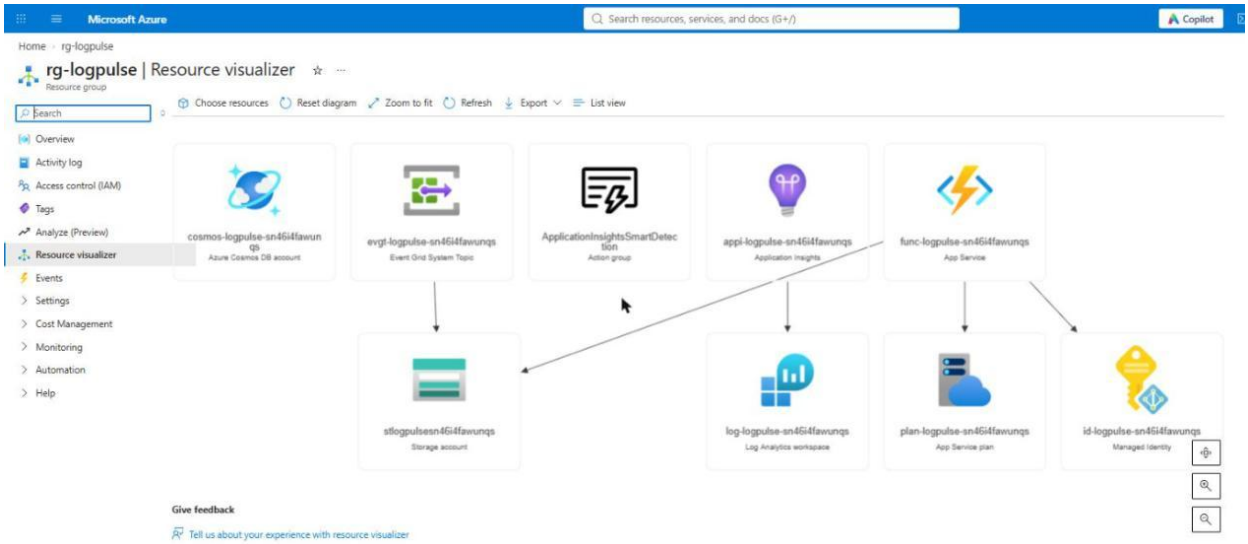


Figure 3: Resource visualizer showing the relationships between the resources

4.1 Problems during deployment

- **Region policy.** The Azure for Students subscription only allows five regions (Belgium Central, Switzerland North, Sweden Central, France Central, Germany West Central). I checked the policy assignment with the Azure CLI and picked Sweden Central, which also supports the Flex Consumption plan.
- **Webhook URL cut off on Windows.** The first attempt to create the Event Grid subscription failed with "Webhook endpoint validation failed … Unauthorized". The URL contains "&code=<key>", and because az is a .cmd file on Windows, cmd.exe treated the & as a command separator and dropped the key. I changed the script to create the subscription with az rest and a JSON body file, so the URL never passes through the command line.
- **Invocation list empty.** The Invocations tab of the function showed zero runs although Event Grid had delivered all events. Querying the Log Analytics workspace showed that the trace logs of ingest_log were there, but the requests table that this tab reads was not filled. I used a KQL query on the traces instead (Figure 13).

5. Testing and results

5.1 Unit tests

The project has 104 automated tests written with pytest. They cover every detection rule with small input snippets, the six sample logs end to end, the test statistics extraction, the storage layer (including the generated Cosmos DB SQL and a fake Cosmos container), the HTTP API, the command-line tool, the local dev server and the Azure Function bindings themselves. A GitHub Actions workflow runs the linter and the tests on every push.

5.2 End-to-end test on Azure

The six sample logs were uploaded to the raw-logs container with scripts/upload-samples.ps1 (Figures 4 and 5).

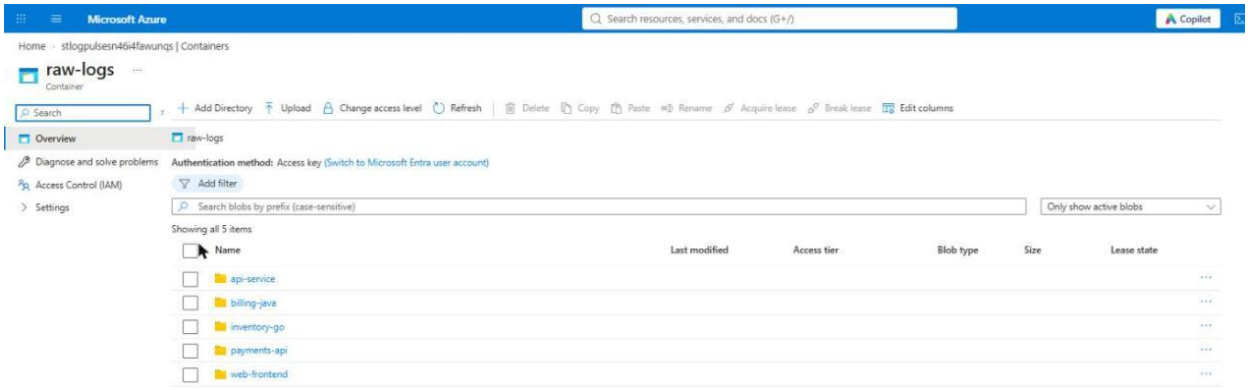


Figure 4: The raw-logs container with one folder per pipeline

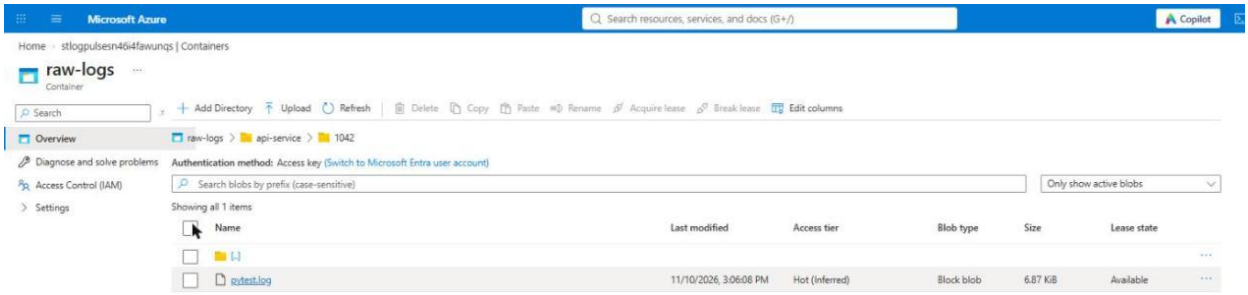


Figure 5: An uploaded log file at raw-logs/api-service/1042/pytest.log

Event Grid published the BlobCreated events and delivered all six of them to the function (Figure 6, Matched Events 6, Delivered Events 6, no failed deliveries).

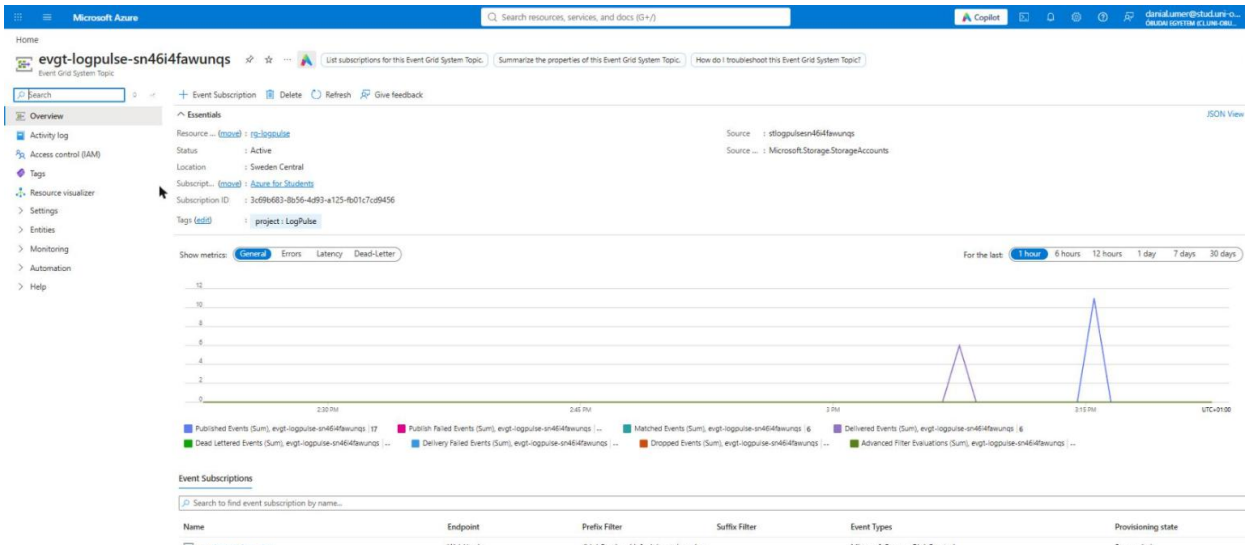


Figure 6: Event Grid system topic with delivered events and the webhook subscription

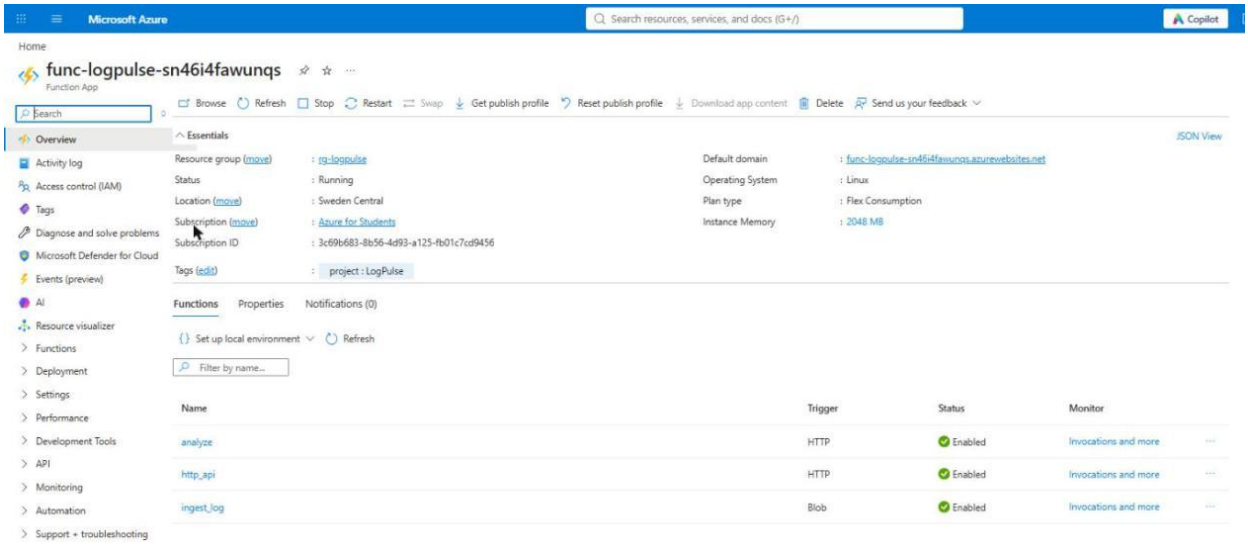


Figure 7: Function App on the Flex Consumption plan with its three functions

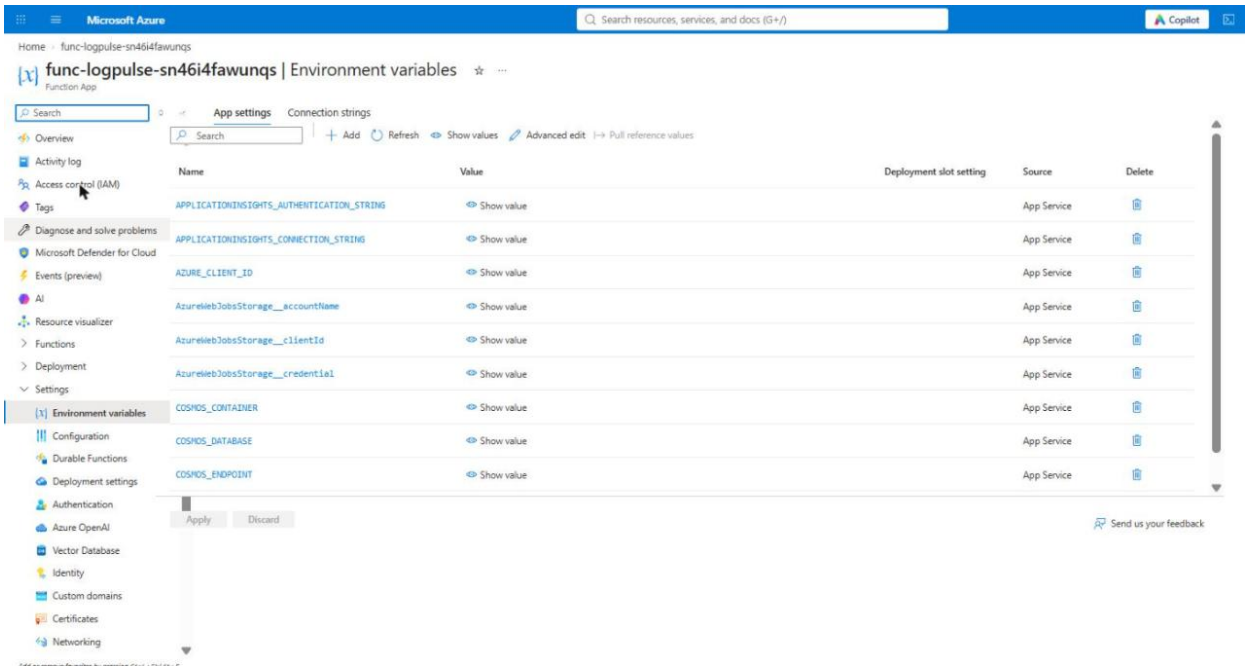


Figure 8: Function App settings: only names and endpoints, no keys or connection strings

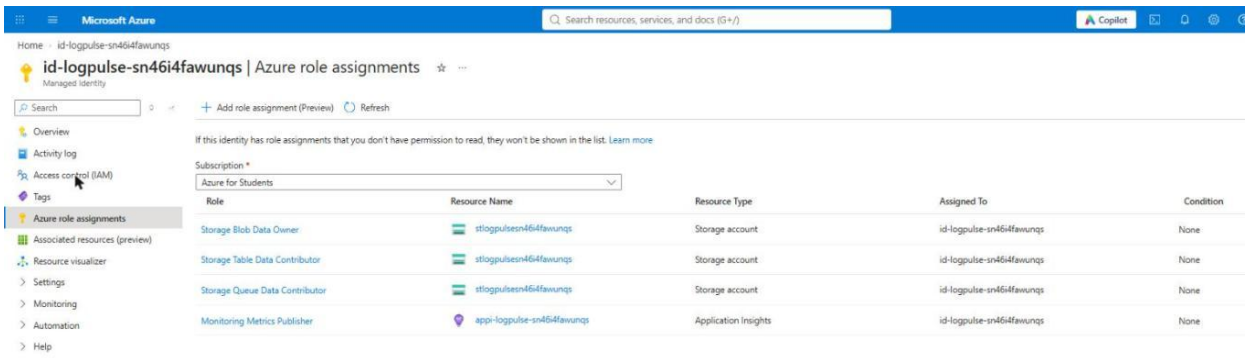


Figure 9: Role assignments of the managed identity

Each upload produced exactly one report in Cosmos DB (Figures 10 to 12). The query in Figure 12 returns the five failed builds.

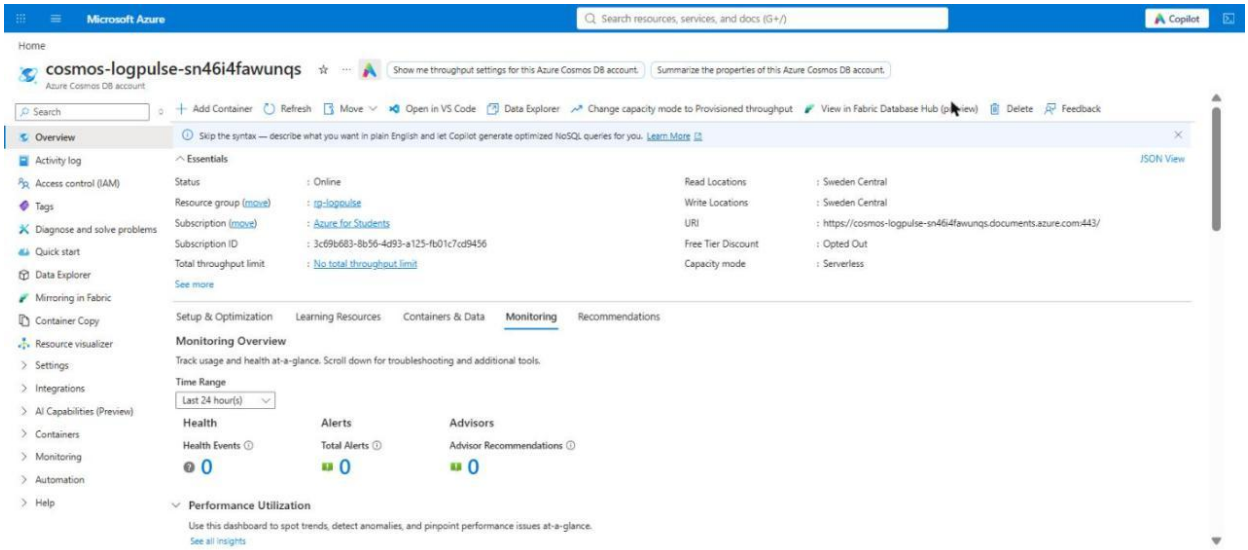


Figure 10: Cosmos DB account in serverless capacity mode

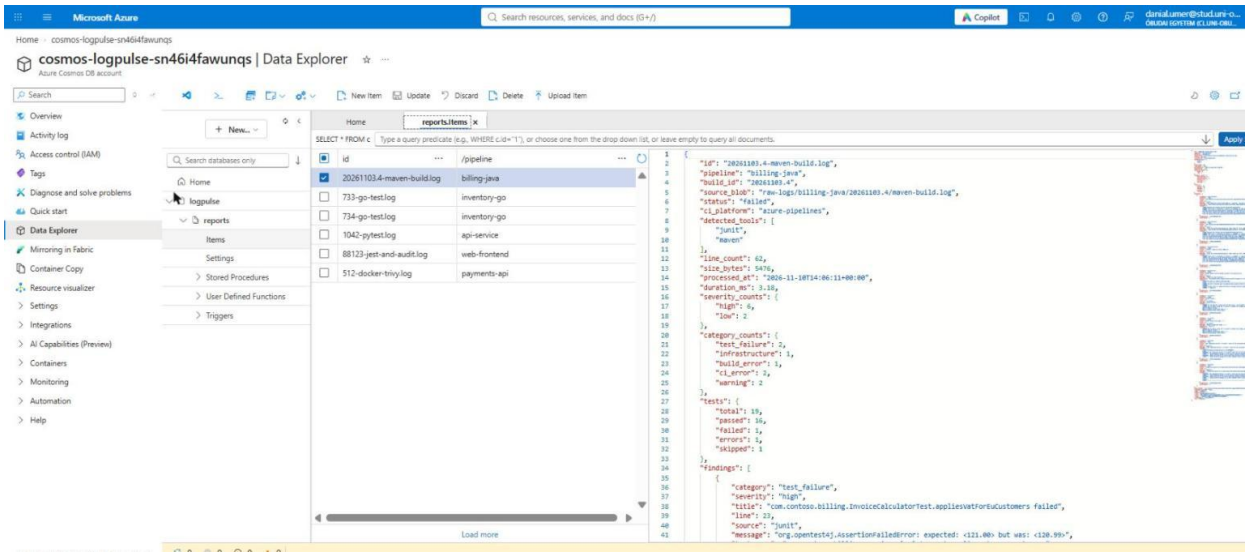


Figure 11: A stored report document in Data Explorer

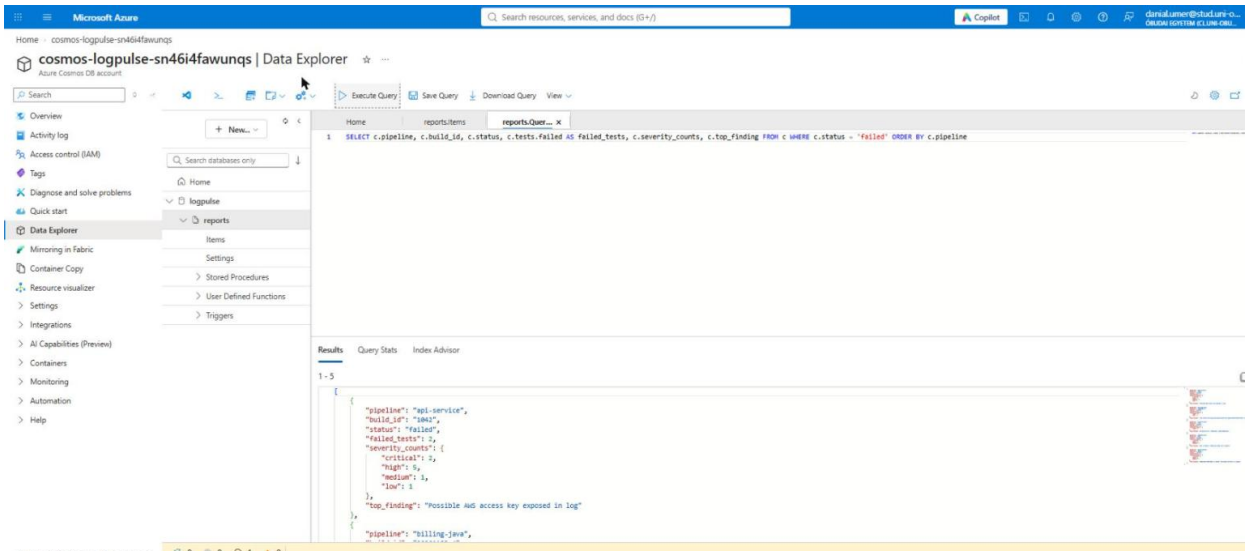


Figure 12: SQL query for all failed builds

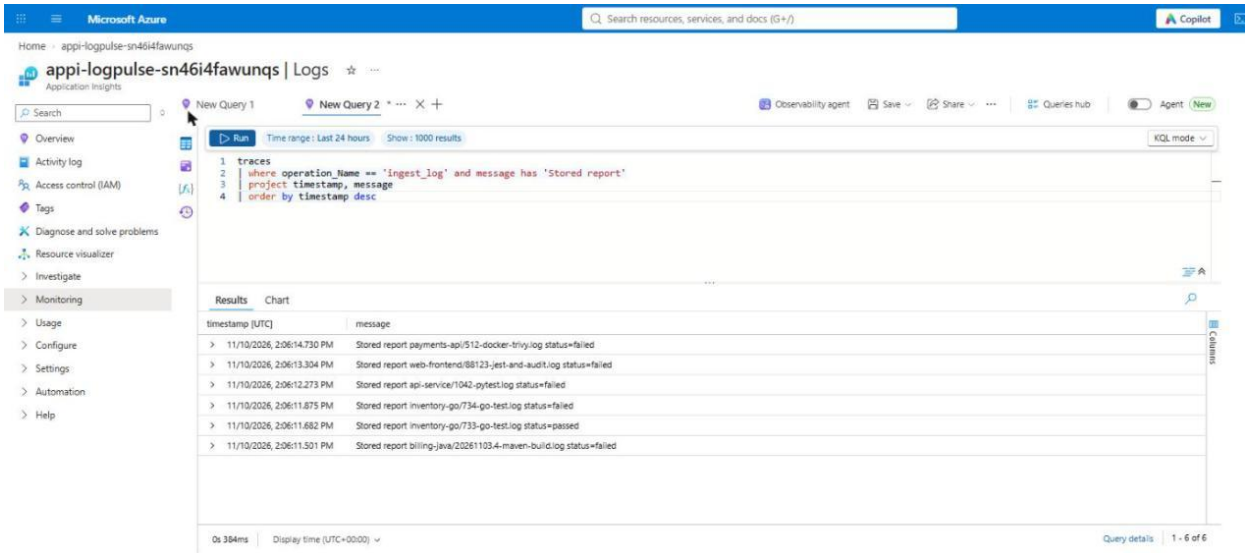


Figure 13: Application Insights traces of ingest_log: one stored report per upload

Table 4 summarises the results for the six sample logs. Parse times are measured inside the function and do not include the blob download or the database write.

Table 4: Results for the six sample logs on Azure

Pipeline / build	CI platform	Lines	Status	Findings (C / H / M / L)	Parse time
api-service #1042	GitHub Actions	97	failed	2 / 5 / 1 / 1	3.5 ms
billing-java #20261103.4	Azure Pipelines	62	failed	0 / 6 / 0 / 2	3.2 ms
inventory-go #733	GitHub Actions	23	passed	0 / 0 / 0 / 1	1.0 ms
inventory-go #734	GitHub Actions	22	failed	0 / 5 / 0 / 0	1.1 ms
payments-api #512	Jenkins	65	failed	2 / 5 / 1 / 0	3.2 ms
web-frontend #88123	GitLab CI	86	failed	2 / 8 / 1 / 0	3.5 ms

I checked every finding against the sample logs by hand. All failed tests, compiler errors, CVEs, npm advisories and the leaked AWS key were found. The passing Go build was correctly marked as passed, with only a deprecation warning. The Maven boilerplate lines such as "Re-run Maven using the -X switch" were not reported as errors, and indented error lines inside Jest failures were attributed to the failing test instead of being reported twice.

5.3 Dashboard

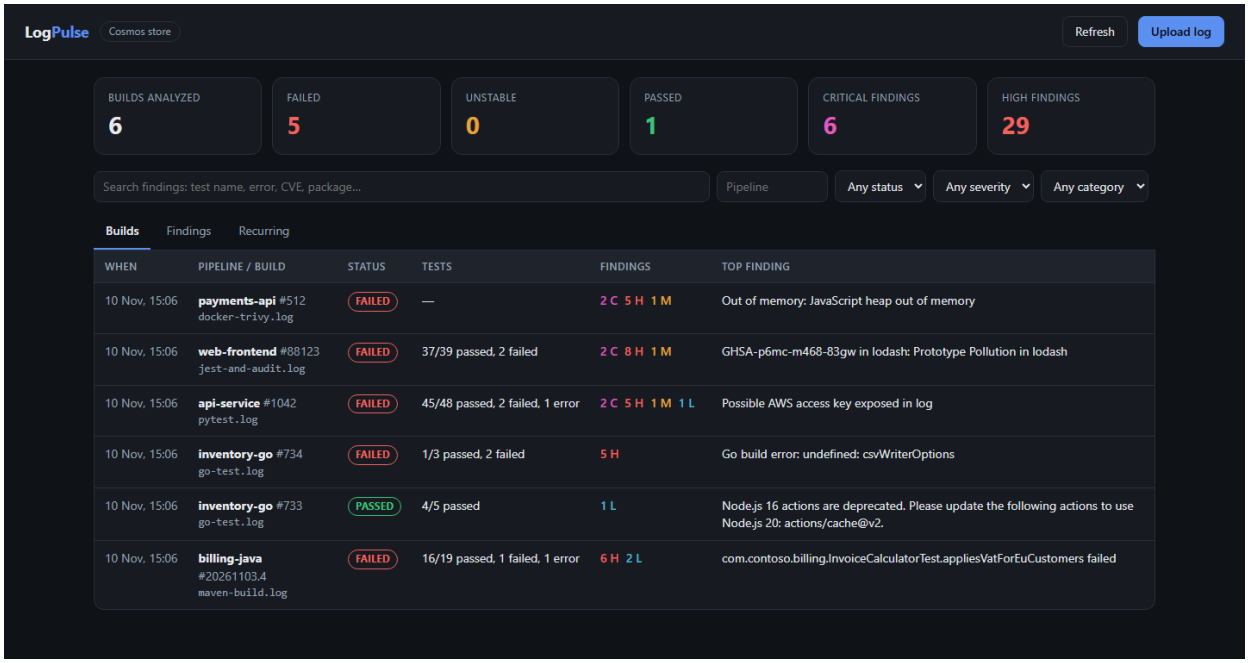


Figure 14: Dashboard: list of analysed builds

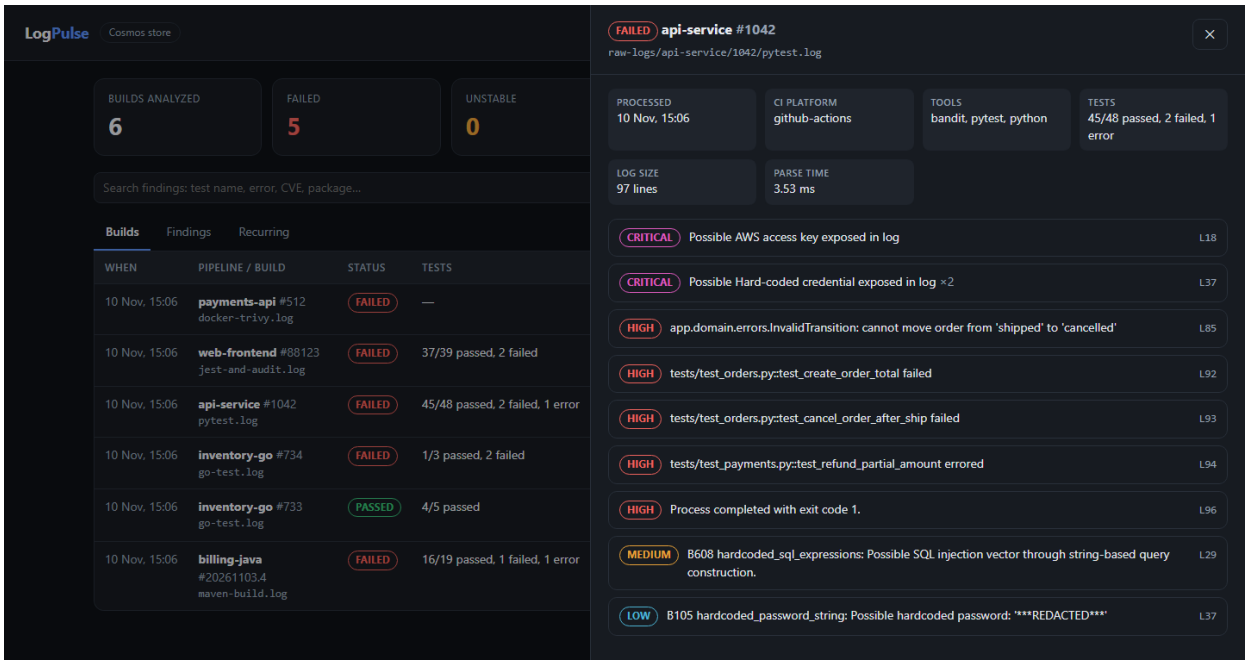


Figure 15: Dashboard: details of build api-service #1042 (the leaked key is redacted)

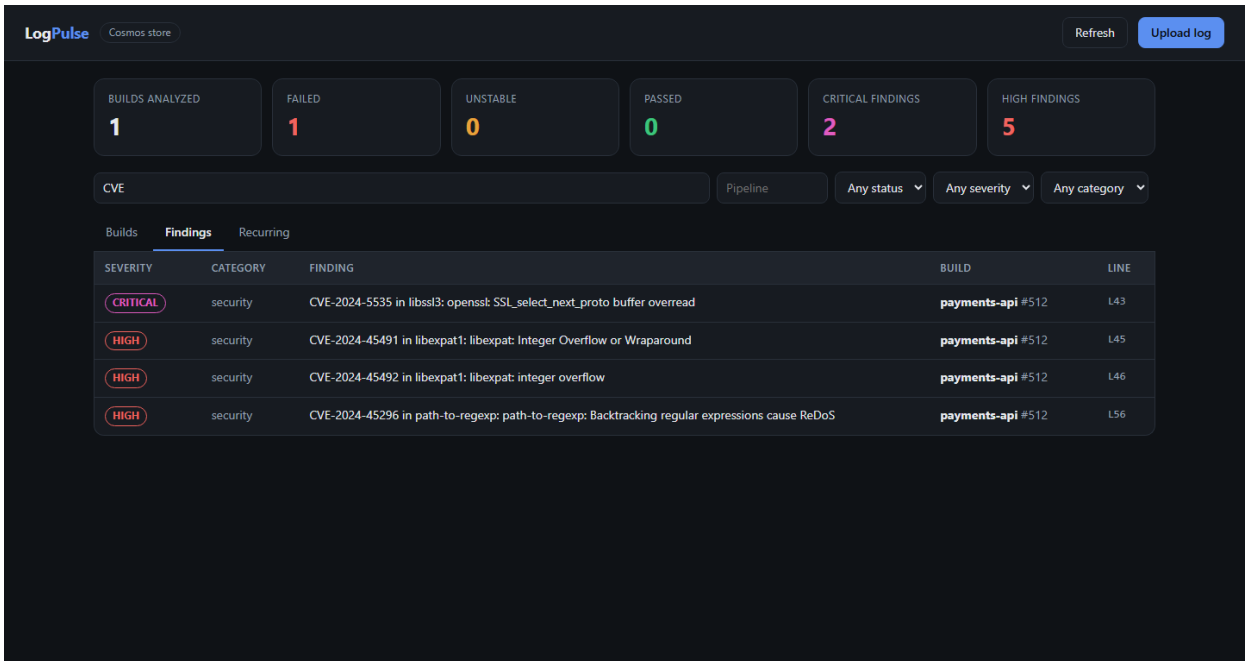


Figure 16: Dashboard: free-text search for "CVE" across all builds

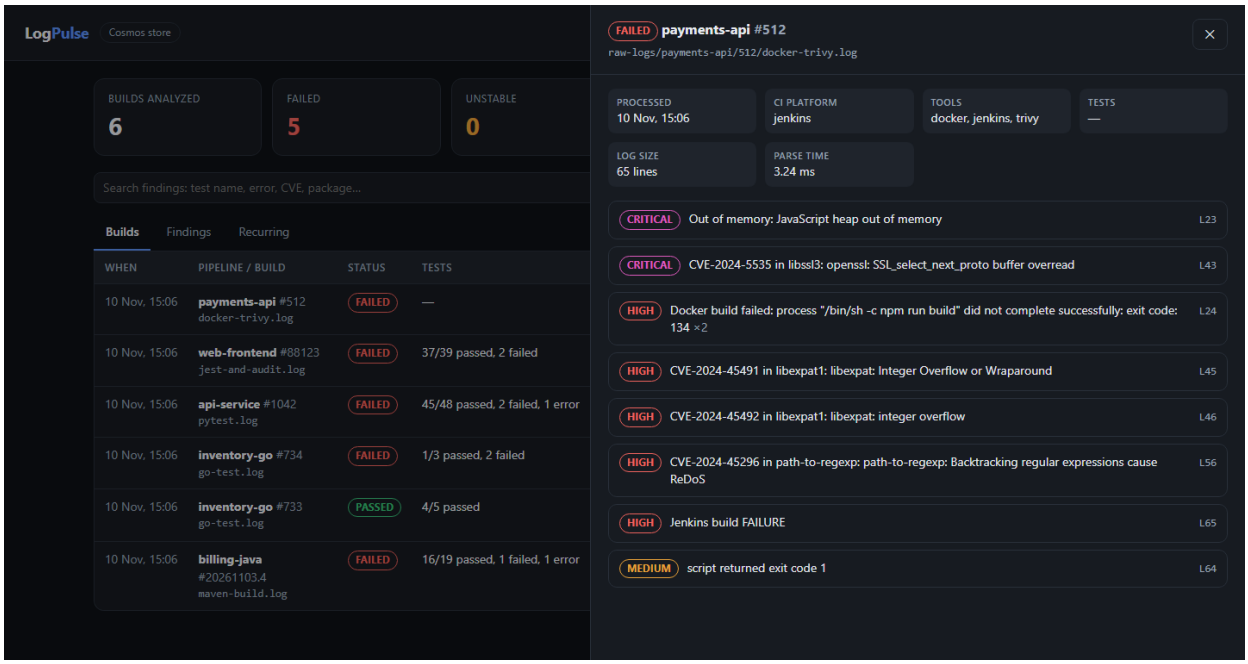


Figure 17: Dashboard: Docker build with an out-of-memory crash and Trivy findings

6. Cost

All services are billed by usage. While no logs are uploaded, the Function App scales to zero and Cosmos DB serverless only charges for stored data, so the idle cost is close to zero. For the testing done in this project (a few dozen function executions and a few hundred database requests) the cost stayed within the free monthly grants and amounted to a few cents of the student credit. The largest item at higher volume would be Cosmos DB request units, which grow with the number of logs and the size of each report.

When the project is no longer needed, scripts/teardown.ps1 deletes the whole resource group.

7. Limitations and future work

- The parser is rule based. Formats it has no rule for are only caught by the generic error and warning rules. Adding a new tool means writing one new rule, which is simple, but it has to be done by hand.
- The read API is anonymous to keep the demo simple. In a real setup it should be protected with Microsoft Entra ID authentication (App Service Authentication).
- Very large logs are read into memory at once. Streaming the blob line by line would lower the memory use.
- There are no alerts yet. A natural next step is an Azure Monitor alert or a Teams/Slack message when a build has critical findings, for example a leaked secret.
- The CI workflow only runs the tests. It could also upload its own log to the raw-logs container after every run so LogPulse analyses its own builds.

8. Conclusion

LogPulse meets the goals set at the start. Uploading a log to Blob Storage is enough to get a structured, searchable summary in Cosmos DB within seconds, without any manual step. The system runs entirely on serverless Azure services, uses a managed identity instead of keys, and is deployed from a single Bicep template. Keeping the parsing logic independent of Azure was the most useful decision. It allowed fast local development and thorough unit testing, and the move to Azure then mostly came down to infrastructure and permissions.

Appendix A: Running the project

Local (no Azure account needed):

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements-dev.txt
python -m pytest                      # 104 tests
python -m logpulse analyze samples    # analyse the sample logs in the terminal
python -m logpulse serve              # dashboard on http://127.0.0.1:7071/api/dashboard
```

Azure:

```
az login
.\scripts\deploy.ps1 -Location swedencentral
.\scripts\upload-samples.ps1
.\scripts\teardown.ps1              # delete everything afterwards
```

Appendix B: Repository structure

Table 5: Repository structure

Path	Content
function_app.py	Azure Functions entry point (blob trigger and HTTP routes)
logpulse/rules.py	Detection rules
logpulse/parser.py	Line cleaning, rule engine, deduplication and fingerprints
logpulse/stats.py	Test count extraction
logpulse/store.py	Cosmos DB and local JSON storage
logpulse/service.py, api.py	Processing, queries and HTTP handlers
logpulse/static/dashboard.html	Dashboard
infra/main.bicep	All Azure resources
scripts/	deploy, upload, teardown and packaging scripts
samples/	Six sample CI logs
tests/	104 pytest tests